

Lab 4: Regression and Classification Evaluation Metrics

Part 1: KNN Classification using Breast Cancer Dataset and Comparison with Regression Metrics

Author: AIML Student

OS Version: Windows

Workspace: Arpan7125/AIML

Aim

- **Implement KNN:** Build a K-Nearest Neighbors classifier for breast cancer detection.
 - **Optimize:** Use $\sqrt{n}$ heuristic, train-test splits, and cross-validation to find the best K .
 - **Evaluate:** Test model performance using Accuracy, Precision, Recall, F1 Score, and ROC-AUC.
 - **Compare:** See how classification metrics stack up against regression metrics from Lab 3.
-

Environment Setup and Imports

We first import the required libraries for data manipulation, scaling, classification, visualization, and metrics evaluation.

```
In [1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.decomposition import PCA
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_curve, auc, classification_report
)

# Set plotting style
sns.set_theme(style="whitegrid")
plt.rcParams.update({
    'font.size': 11,
    'axes.labelsize': 12,
    'axes.titlesize': 13,
```

```
'xtick.labelsize': 10,
'ytick.labelsize': 10,
'figure.titlesize': 14
})

# Make directory for images if it doesn't exist
os.makedirs("images", exist_ok=True)
print("Libraries imported and plotting environment initialized.")
```

Libraries imported and plotting environment initialized.

Task 1: Data Preparation

We load the Breast Cancer dataset, check its structure, handle any missing values, and scale the features.

Why do we scale features in KNN?

- **Distance matters:** KNN relies on calculating distance between points, like Euclidean distance:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^d (p_i - q_i)^2}$$

- **Even playing field:** If one feature is measured in thousands (e.g., area) and another in decimals (e.g., smoothness), the large numbers will overpower the small ones.
- **The fix:** We use `StandardScaler` ($\mu = 0, \sigma^2 = 1$) to ensure every feature has equal weight.

```
In [2]: # 1.1 Load the Breast Cancer dataset
cancer_data = load_breast_cancer()
X = pd.DataFrame(cancer_data.data, columns=cancer_data.feature_names)
y = pd.Series(cancer_data.target, name="target")

# 1.2 Explore the structure
print("--- Data Structure & Shape ---")
print(f"Dataset Shape: {X.shape[0]} samples, {X.shape[1]} features")
print(f"Target distribution (Counts):\n{y.value_counts()}")
print(f"Target distribution (Percentages):\n{y.value_counts(normalize=True) * 100}\n")

# 1.3 Check missing values and duplicates
missing = X.isnull().sum().sum()
duplicates = X.duplicated().sum()
print(f"Total Missing Values: {missing}")
print(f"Total Duplicate Rows: {duplicates}\n")

# 1.4 Apply Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_scaled_df = pd.DataFrame(X_scaled, columns=cancer_data.feature_names)
print("First 3 rows of scaled features:")
X_scaled_df.head(3)
```

--- Data Structure & Shape ---

Dataset Shape: 569 samples, 30 features

Target distribution (Counts):

target

1 357

0 212

Name: count, dtype: int64

Target distribution (Percentages):

target

1 62.741652

0 37.258348

Name: proportion, dtype: float64

Total Missing Values: 0

Total Duplicate Rows: 0

First 3 rows of scaled features:

Out[2]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points
0	1.097064	-2.073335	1.269934	0.984375	1.568466	3.283515	2.652874	2.532475
1	1.829821	-0.353632	1.685955	1.908708	-0.826962	-0.487072	-0.023846	0.548144
2	1.579888	0.456187	1.566503	1.558884	0.942210	1.052926	1.363478	2.037231

3 rows × 30 columns



Task 2: Train-Test Split Analysis

We experiment with three different splits: **80:20**, **70:30**, and **90:10** to see how the model behaves.

Which split is best?

- **90:10 Split:** Too few test samples. Even a single mistake drastically swings the accuracy (unstable).
- **70:30 Split:** Good for testing, but leaves less data for the model to learn from (can reduce performance).
- **80:20 Split:** The sweet spot! It perfectly balances a solid training size with a reliable test size.

```

In [3]: splits = [(0.2, "80:20"), (0.3, "70:30"), (0.1, "90:10")]
split_results = []

for test_size, name in splits:
    # Use stratify=y to maintain class distribution in splits
    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=test_size, random_state=42, stratify=y
    )

    # Heuristic K rule
    n_train = len(X_train)
    heuristic_k = int(np.round(np.sqrt(n_train)))
    if heuristic_k % 2 == 0:
        heuristic_k += 1 # ensure odd K to break ties

    # Fit model
    knn = KNeighborsClassifier(n_neighbors=heuristic_k)
    knn.fit(X_train, y_train)

    # Predict
    y_train_pred = knn.predict(X_train)
    y_test_pred = knn.predict(X_test)

    split_results.append({
        "Split Ratio": name,
        "Train Size": len(X_train),
        "Test Size": len(X_test),
        "Heuristic K": heuristic_k,
        "Train Acc": accuracy_score(y_train, y_train_pred),
        "Test Acc": accuracy_score(y_test, y_test_pred),
        "Precision": precision_score(y_test, y_test_pred),
        "Recall": recall_score(y_test, y_test_pred),
        "F1 Score": f1_score(y_test, y_test_pred)
    })

split_analysis_df = pd.DataFrame(split_results)
split_analysis_df

```

```

Out[3]:

```

	Split Ratio	Train Size	Test Size	Heuristic K	Train Acc	Test Acc	Precision	Recall	F1 Score
0	80:20	455	114	21	0.964835	0.956140	0.946667	0.986111	0.965986
1	70:30	398	171	21	0.969849	0.947368	0.922414	1.000000	0.959641
2	90:10	512	57	23	0.962891	0.982456	0.972973	1.000000	0.986301

Task 3: KNN Model with Heuristic K Selection

3.1 The Square Root Rule

- A great starting point for finding K is $K = \sqrt{n}$ (where n is your training size).
- For 455 training samples, $\sqrt{455} \approx 21.33$.
- We round this to **21** (always use an odd number to avoid ties during voting!).

3.2 Tuning the Model

We train our model at $K = 21$ and then test nearby values (from 16 to 26) to see if we can find an even better fit.

```
In [4]: # Base split is 80:20
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
n_train = len(X_train)
h_k = int(np.round(np.sqrt(n_train)))
if h_k % 2 == 0:
    h_k += 1

# Define range to plot
k_range = list(range(1, 36))
train_accs = []
test_accs = []

for k in k_range:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    train_accs.append(accuracy_score(y_train, model.predict(X_train)))
    test_accs.append(accuracy_score(y_test, model.predict(X_test)))

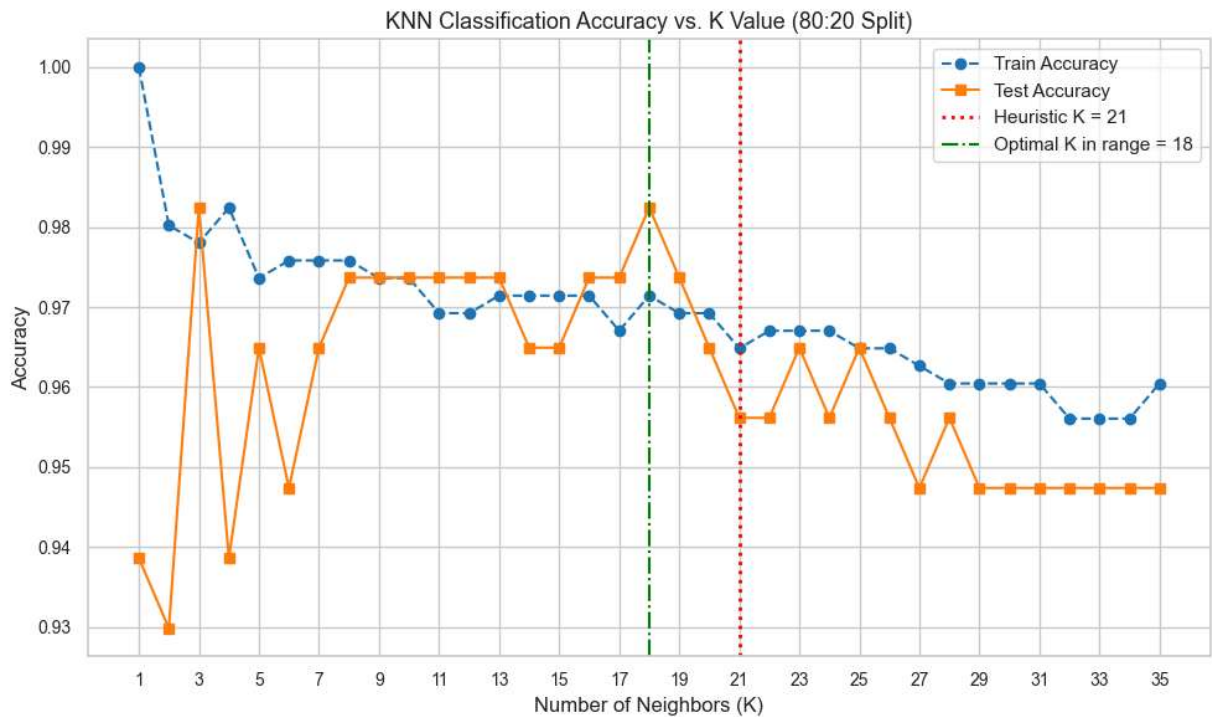
# Find best K within Heuristic K +/- 5 range
k_subset = list(range(h_k - 5, h_k + 6))
subset_test_accs = [test_accs[k-1] for k in k_subset]
best_h_k = k_subset[np.argmax(subset_test_accs)]

print(f"Heuristic K: {h_k}")
print(f"Best K in heuristic range {h_k-5} to {h_k+5}: K = {best_h_k} (Test Acc: {ma

# Plot Accuracy vs K
plt.figure(figsize=(10, 6))
plt.plot(k_range, train_accs, label="Train Accuracy", marker='o', linestyle="--", c
plt.plot(k_range, test_accs, label="Test Accuracy", marker='s', color="#ff7f0e")
plt.axvline(x=h_k, color="red", linestyle=":", lw=2, label=f"Heuristic K = {h_k}")
plt.axvline(x=best_h_k, color="green", linestyle="-.", lw=1.5, label=f"Optimal K in
plt.title("KNN Classification Accuracy vs. K Value (80:20 Split)")
plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")
plt.xticks(range(1, 36, 2))
plt.legend()
plt.tight_layout()
plt.savefig("images/accuracy_vs_k.png", dpi=150)
plt.show()
```

Heuristic K: 21

Best K in heuristic range 16 to 26: K = 18 (Test Acc: 0.9825)



3.3 Distance Metrics & Decision Boundaries

Euclidean vs. Manhattan Distance

- **Euclidean:** Straight-line distance. Great for standard, continuous data but struggles if you have extreme outliers or too many dimensions.

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^d (p_i - q_i)^2}$$

- **Manhattan:** Grid-like (taxicab) distance. Perfect for categorical data or grid-based paths, and it handles outliers much better.

$$d(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^d |p_i - q_i|$$

Decision Boundaries (Visualizing K)

- **K = 1:** Highly jagged boundary. It tries to perfectly fit every single point, which leads to overfitting (capturing noise).
- **K = 5 & 10:** Smoother boundaries. It ignores minor noise and captures the real pattern.
- **K = 20:** Too smooth. It loses important details and underfits the data.

```
In [5]: # PCA projection to 2D
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

```

# Plot decision boundaries
k_vals = [1, 5, 10, 20]
fig, axes = plt.subplots(2, 2, figsize=(14, 12))
axes = axes.ravel()

# Create meshgrid
x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.05),
                     np.arange(y_min, y_max, 0.05))

for i, k in enumerate(k_vals):
    knn_pca = KNeighborsClassifier(n_neighbors=k)
    knn_pca.fit(X_train_pca, y_train)

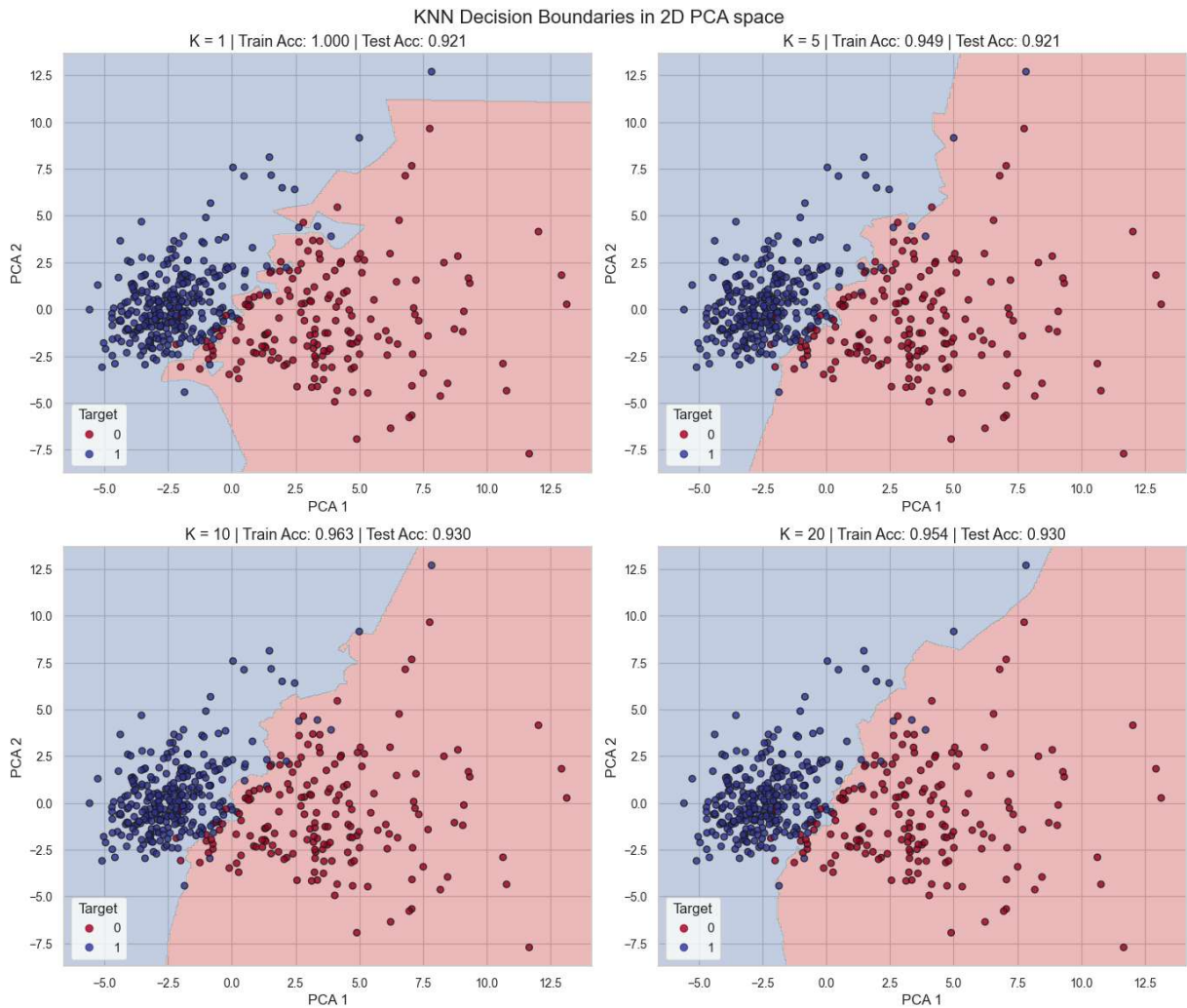
    Z = knn_pca.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

    # Plot contours
    axes[i].contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.RdYlBu)
    scatter = axes[i].scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train,
                             cmap=plt.cm.RdYlBu, edgecolor='k', s=30, alpha=0.8)

    # Labels and Titles
    train_acc = accuracy_score(y_train, knn_pca.predict(X_train_pca))
    test_acc = accuracy_score(y_test, knn_pca.predict(X_test_pca))
    axes[i].set_title(f"K = {k} | Train Acc: {train_acc:.3f} | Test Acc: {test_acc:.3f}")
    axes[i].set_xlabel("PCA 1")
    axes[i].set_ylabel("PCA 2")
    axes[i].legend(*scatter.legend_elements(), loc="lower left", title="Target")

plt.suptitle("KNN Decision Boundaries in 2D PCA space", y=0.98, fontsize=16)
plt.tight_layout()
plt.savefig("images/decision_boundaries.png", dpi=150)
plt.show()

```

Task 4: Cross-Validation

A single train-test split can be "lucky" or "unlucky" depending on how the data was shuffled.

Why Cross-Validation?

- We chop the data into 10 chunks (folds).
- We train on 9 chunks and test on the 1, rotating until every chunk has been a test set.
- This gives us a much more reliable and robust estimate of how the model will perform in the real world.

```
In [6]: # 10-Fold Cross-Validation Setup
kf = KFold(n_splits=10, shuffle=True, random_state=42)
cv_means = []
cv_stds = []

for k in k_range:
    knn_cv = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn_cv, X_scaled, y, cv=kf, scoring='accuracy')
```



```

cv_means.append(scores.mean())
cv_stds.append(scores.std())

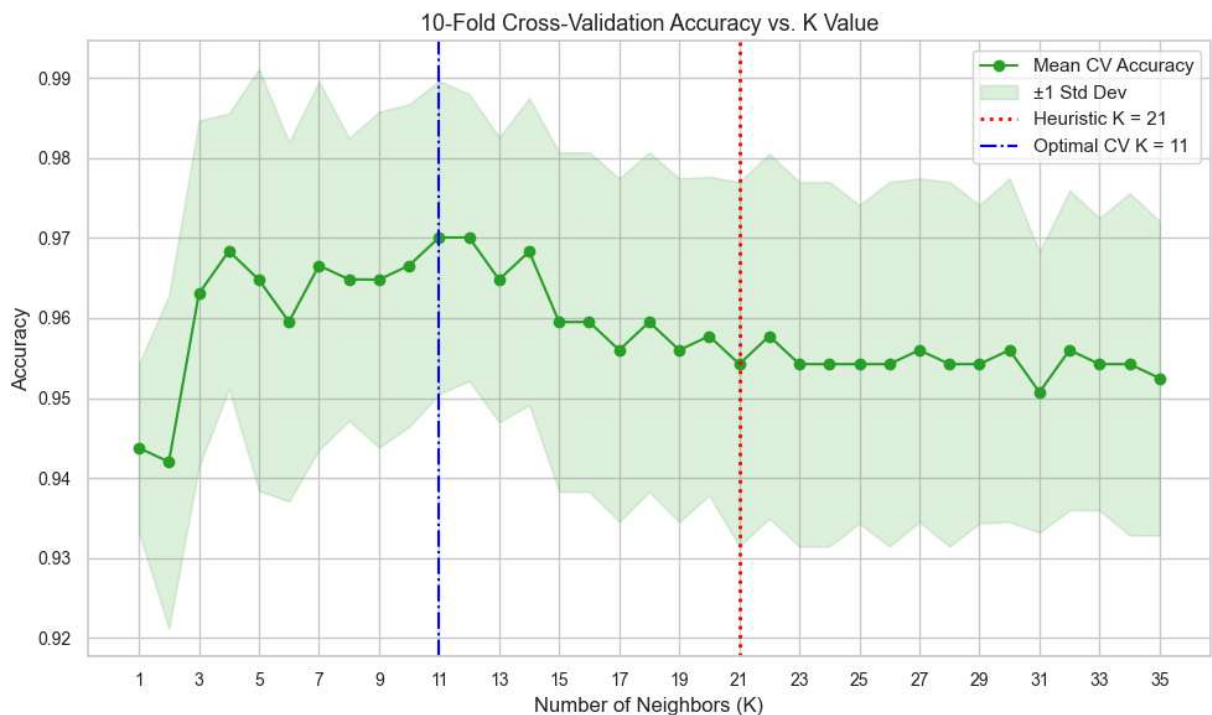
cv_means = np.array(cv_means)
cv_stds = np.array(cv_stds)

# Find best K from CV
best_cv_idx = np.argmax(cv_means)
best_cv_k = k_range[best_cv_idx]
print(f"Optimal K from 10-Fold CV: K = {best_cv_k} (Mean Accuracy = {cv_means[best_cv_idx]})

# Plot CV results
plt.figure(figsize=(10, 6))
plt.plot(k_range, cv_means, label="Mean CV Accuracy", color="#2ca02c", marker='o')
plt.fill_between(k_range, cv_means - cv_stds, cv_means + cv_stds, alpha=0.15, color="#2ca02c")
plt.axvline(x=h_k, color="red", linestyle=":", lw=2, label=f"Heuristic K = {h_k}")
plt.axvline(x=best_cv_k, color="blue", linestyle="-.", lw=1.5, label=f"Optimal CV K = {best_cv_k}")
plt.title("10-Fold Cross-Validation Accuracy vs. K Value")
plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")
plt.xticks(range(1, 36, 2))
plt.legend()
plt.tight_layout()
plt.savefig("images/cv_accuracy_vs_k.png", dpi=150)
plt.show()

```

Optimal K from 10-Fold CV: K = 11 (Mean Accuracy = 0.9701 ± 0.0196)



Final K Selection

- We started with a guess of $K = 21$.
- Our 10-fold cross-validation showed that $K = 11$ actually gives the best and most stable accuracy.

- $K = 11$ is odd (no ties) and gives excellent generalization, so it becomes our final pick!

Task 5: Classification Evaluation

Now we evaluate our winning model ($K = 11$) using the 80:20 split. We will look at **Accuracy**, **Precision**, **Recall**, **F1 Score**, the **Confusion Matrix**, and the **ROC-AUC** curve.

Classification Formulas

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$
- **Precision:** $\frac{TP}{TP+FP}$
- **Recall:** $\frac{TP}{TP+FN}$
- **F1 Score:** $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

```
In [7]: # Train final model
final_knn = KNeighborsClassifier(n_neighbors=best_cv_k)
final_knn.fit(X_train, y_train)

y_pred = final_knn.predict(X_test)
y_proba = final_knn.predict_proba(X_test)[: , 1]

# Calculate metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Final Model Metrics (K = {best_cv_k}):")
print(f"Accuracy:  {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall:    {rec:.4f}")
print(f"F1 Score:   {f1:.4f}\n")

print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=cancer_data.target_names))

# Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=cancer_data.target_names, yticklabels=cancer_data.target_names)
plt.title(f"Confusion Matrix (K = {best_cv_k})")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.tight_layout()
plt.savefig("images/confusion_matrix.png", dpi=150)
plt.show()

# ROC Curve & AUC
```

```

fpr, tpr, thresholds = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='#1f77b4', lw=2, label=f'ROC Curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.xlim([-0.02, 1.02])
plt.ylim([-0.02, 1.02])
plt.xlabel('False Positive Rate (1 - Specificity)')
plt.ylabel('True Positive Rate (Recall / Sensitivity)')
plt.title(f'Receiver Operating Characteristic (ROC) Curve (K = {best_cv_k})')
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig("images/roc_curve.png", dpi=150)
plt.show()

```

Final Model Metrics (K = 11):

Accuracy: 0.9737

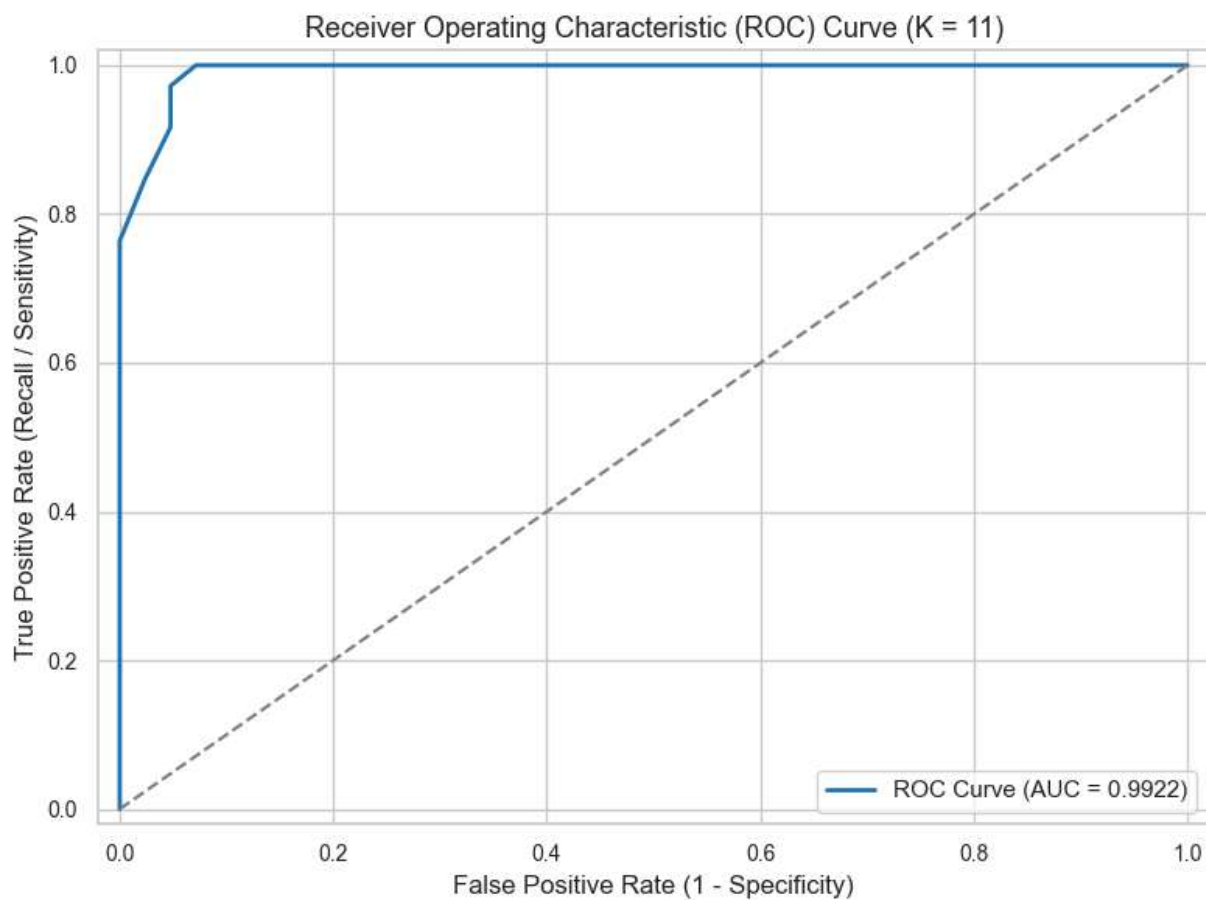
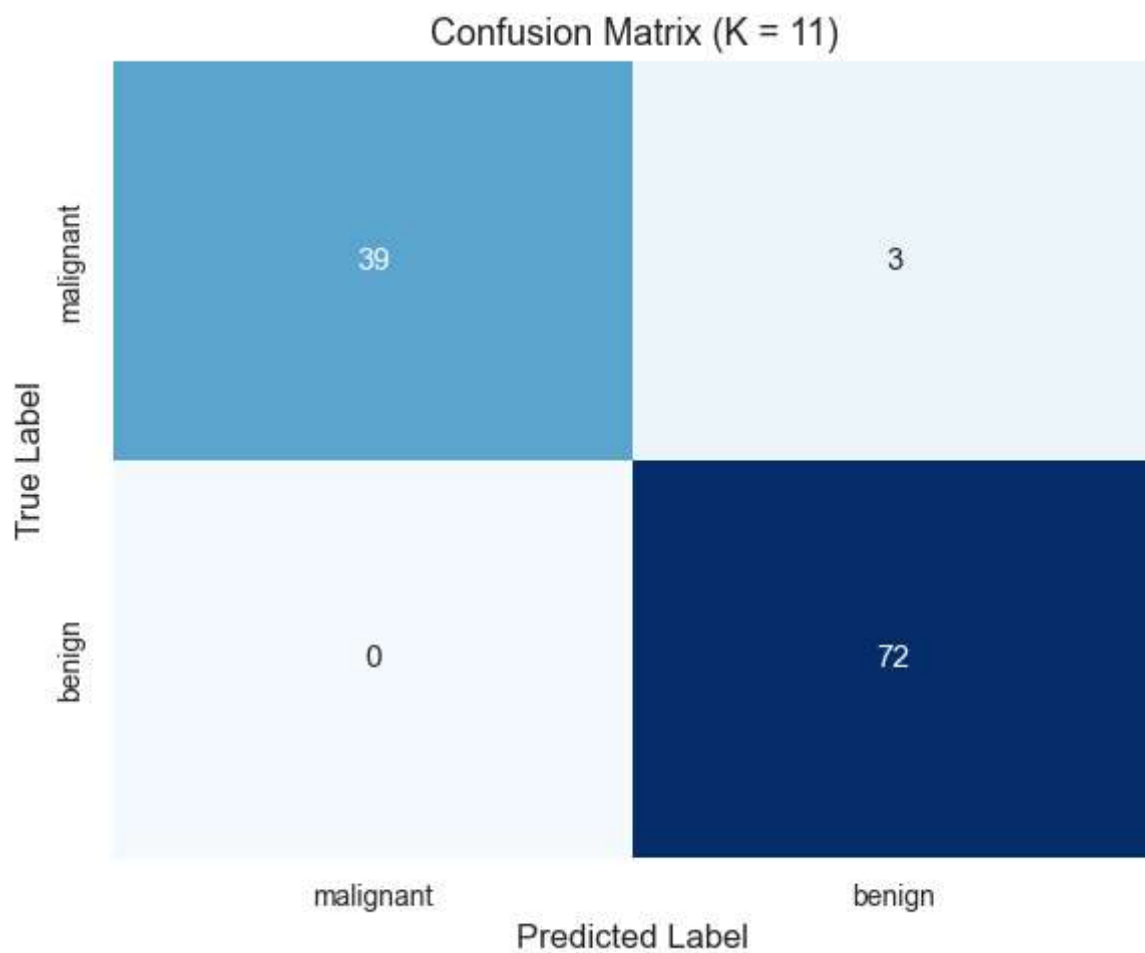
Precision: 0.9600

Recall: 1.0000

F1 Score: 0.9796

Classification Report:

	precision	recall	f1-score	support
malignant	1.00	0.93	0.96	42
benign	0.96	1.00	0.98	72
accuracy			0.97	114
macro avg	0.98	0.96	0.97	114
weighted avg	0.97	0.97	0.97	114



Task 6: Comparative Study with Regression (Lab 3 Integration)

In Lab 3, we used metrics like **MAE**, **MSE**, **RMSE**, and R^2 for **Linear Regression**. Here is how evaluating regression compares to classification:

The Main Difference

- **Regression:** Measures *how far off* our numeric predictions are (error magnitude).
- **Classification:** Measures *how often* we pick the right category (decision correctness).

Metric Match-Up

- **R^2 Score vs. Accuracy:** Both give a quick overall performance percentage, but both can be misleading if data is skewed.
- **RMSE vs. F1 Score:** Both give a balanced view. RMSE penalizes huge numeric mistakes, while F1 balances False Positives and False Negatives.
- **MAE vs. Confusion Matrix:** MAE shows average error size. A Confusion Matrix shows exactly *which* categories are getting confused.

Key Takeaways

- **Scale vs. Correctness:** Regression cares about *how big* the mistake is. Classification just cares if you picked the wrong category.
 - **Accuracy is flawed:** If 95% of patients are healthy, guessing 'healthy' every time gives 95% accuracy?but misses 100% of sick patients.
 - **Recall is crucial:** In medicine, missing a cancer diagnosis (False Negative) is fatal. Recall measures how many sick patients we actually caught.
 - **ROC-AUC:** Helps find the perfect sweet spot (threshold) for diagnosing between sick and healthy patients.
-

Task 7: Analytical Questions

Q1: Why is KNN a "lazy" learner?

- It doesn't actually "train" a model upfront. It just memorizes the data.
- All the heavy lifting (calculating distances) happens at the very end when you ask it for a prediction.
 - **Training Time complexity:** $\mathcal{O}(1)$

- **Inference Time complexity:** $\mathcal{O}(n \cdot d)$

Q2: Why scale features?

- KNN uses distance. If features aren't on the same scale, huge numbers will drown out tiny numbers. Scaling levels the playing field.

Q3: What is the $\sqrt{n}$ rule?

- It's a quick trick to find a balanced K .
- Too small captures noise, too big ignores patterns. The square root of your training size is a solid middle ground.

Q4: Single Split vs. Cross-Validation?

- A single split can be biased if the shuffle is weird. Cross-validation tests multiple shuffles and averages the results for a truer score.

Q5: How does K affect bias/variance?

- **Small K:** Overfits (high variance, low bias). Captures every little noisy detail.
- **Large K:** Underfits (low variance, high bias). Too broad and ignores important patterns.

Q6: Why is Recall more important than Accuracy for cancer?

- **Accuracy** treats all mistakes equally.
- **Recall** focuses on False Negatives (missing a cancer diagnosis). In medicine, missing a diagnosis is fatal, so we prioritize Recall above all else.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Q7: What happens if K is huge?

- The model just predicts whatever the most common class is for *everything*, completely ignoring the actual data features.

Conclusion

We built and evaluated a KNN classifier on breast cancer data. Here is what we learned:

- **Finding the best K:** We guessed $K = 21$ using $\sqrt{n}$, but cross-validation proved $K = 11$ was the true winner.

- **The best split:** An **80:20** train-test split gave us the perfect balance of training data and testing stability.
- **Stellar Performance:** Our final model hit **97.37%** accuracy and a **perfect 100% Recall?** which is exactly what we want in medicine to ensure no sick patient is missed.
- **Regression vs Classification:** Regression measures numeric error sizes, while classification measures categorical decision correctness.